

Restructured ML Implementation Plan: Hybrid B2B2C + B2C

CarbonSense AI - Technical Roadmap After TEME Completion

Status: TEME Engine COMPLETE **Approach:** Hybrid (80% B2B2C employer-sponsored, 20% B2C premium) **Focus:** Technical engineering excellence, NOT business strategy

CRITICAL STRATEGIC CHANGES

What Changed from Previous Plan

BEFORE (Pure B2C Focus): - Individual user features only - Single-user ML pipelines - Personal dashboards - No multi-tenant considerations

NOW (Hybrid B2B2C + B2C): - **Same ML models** (no duplication - 70% code reuse) - **Added:** Organization-level aggregation - **Added:** Bulk processing for HR/admin - **Added:** Multi-tenant isolation - **Added:** Team/department analytics - **Priority:** Features that serve BOTH individuals AND organizations

Scope Philosophy: - Keep: All original ML models (OCR, Food, Policy, Forecasting, XAI, RL) - Add: Multi-user outputs, organization analytics APIs - Don't add: Complex business logic, admin UIs, billing systems - Don't add: New ML models beyond original plan

Engineering Principle: > "One ML pipeline → Multiple output formats (individual + organization)" > Not "Two separate ML systems"

UPDATED ML ROADMAP (TECHNICAL TRACKS)

TEME Engine	DONE	
	↓	
TRACK 1: Receipt OCR	(Weeks 7-8)	← Multi-user + Bulk
TRACK 2: Food Recognition	(Weeks 9-10)	← Cafeteria + Teams
TRACK 3: Policy NLP	(Weeks 9-12)	← Already B2B-focused
TRACK 4: Forecasting + Analytics	(Weeks 13-16)	← Org-level + XAI
TRACK 5: Multi-Tenant ML Infra	(Weeks 17-18)	← NEW: Isolation layer
TRACK 6: RL Behavioral Nudging	(Weeks 19-20)	← Team-aware
TRACK 7: Admin Analytics APIs	(Semester 2)	← Aggregations

What's New: - **Track 5:** Multi-tenant infrastructure (data isolation, role-based access) - **Track 7:** Organization-level analytics APIs (for HR dashboards) - **All tracks:** Now output both individual AND organization-level data

TRACK 1: Receipt OCR Pipeline (Enhanced for B2B2C)

Original Features (Kept)

- Image preprocessing + dual OCR (Tesseract + EasyOCR)
- BERT NER for field extraction
- Fuzzy carbon mapping
- Individual receipt processing

New B2B2C Features (Added)

1.1 Bulk Receipt Processing API Use Case: HR uploads zip file of 500 employee expense receipts

Technical Implementation:

```
# ml_services/ocr/bulk_processor.py
"""
Process multiple receipts in batch for organization-wide upload.
Parallelize using multiprocessing for 10x speedup.
"""

import multiprocessing as mp
from pathlib import Path
from typing import List, Dict
import zipfile, tempfile, shutil
from concurrent.futures import import ProcessPoolExecutor, as_completed

from .text_extractor import extract_text
from .carbon_mapper import calculate_receipt_carbon, load_carbon_database

MAX_WORKERS = 4 # CPU cores to use

def process_receipt_batch(
    receipt_paths: List[str],
    organization_id: str,
    uploaded_by_user_id: str
) -> Dict:
    """
    Process multiple receipts in parallel.

    Returns:
    {
        "total_receipts": 500,
```

```

        "processed": 487,
        "failed": 13,
        "total_carbon_kg": 1250.5,
        "receipts": [...],
        "organization_summary": {...}
    }
    """
    carbon_db = load_carbon_database()
    results = []
    failed = []

    # Parallel processing
    with ProcessPoolExecutor(max_workers=MAX_WORKERS) as executor:
        futures = {
            executor.submit(process_single_receipt, path, carbon_db): path
            for path in receipt_paths
        }

        for future in as_completed(futures):
            path = futures[future]
            try:
                result = future.result()
                result['uploaded_by'] = uploaded_by_user_id
                result['organization_id'] = organization_id
                results.append(result)
            except Exception as e:
                failed.append({"path": path, "error": str(e)})

    # Organization-level aggregation
    total_carbon = sum(r.get('carbon_analysis', {}).get('total_carbon_kg', 0)
                       for r in results)

    # Category breakdown (for org dashboard)
    category_breakdown = aggregate_by_category(results)

    # Department breakdown (if user has department info)
    dept_breakdown = aggregate_by_department(results)

    return {
        "total_receipts": len(receipt_paths),
        "processed": len(results),
        "failed": len(failed),
        "failed_receipts": failed,
        "total_carbon_kg": round(total_carbon, 3),
        "receipts": results,
    }

```

```

        # Organization-level analytics
        "organization_summary": {
            "organization_id": organization_id,
            "category_breakdown": category_breakdown, # {food: 450kg, transport: 300kg, ..}
            "department_breakdown": dept_breakdown, # {engineering: 200kg, sales: 150kg, ..}
            "average_per_receipt": round(total_carbon / max(len(results), 1), 3),
            "top_carbon_categories": get_top_categories(category_breakdown, n=5)
        }
    }

def process_single_receipt(image_path: str, carbon_db: dict) -> dict:
    """Process one receipt - same as before."""
    extraction = extract_text(image_path)

    # Parse items (simplified - use NER in production)
    import re
    lines = extraction['text'].split('\n')
    items = []
    for line in lines:
        if re.search(r'\d+\.\d{2}', line):
            name = re.sub(r'[\d.]+\$', '', line).strip()
            if len(name) > 2:
                items.append({'name': name})

    carbon_result = calculate_receipt_carbon(items, carbon_db)

    return {
        "image_path": image_path,
        "ocr_method": extraction['method'],
        "carbon_analysis": carbon_result
    }

def aggregate_by_category(results: List[dict]) -> dict:
    """Aggregate carbon by category across all receipts."""
    categories = {}
    for receipt in results:
        for item in receipt.get('carbon_analysis', {}).get('mapped_items', []):
            cat = item.get('category', 'unknown')
            categories[cat] = categories.get(cat, 0) + item.get('carbon_kg', 0)
    return {k: round(v, 3) for k, v in categories.items()}

def aggregate_by_department(results: List[dict]) -> dict:
    """Aggregate by department if user metadata available."""

```

```

# Would look up user's department from database
# Simplified here - in production, join with user table
departments = {}
for receipt in results:
    dept = receipt.get('user_department', 'unassigned')
    carbon = receipt.get('carbon_analysis', {}).get('total_carbon_kg', 0)
    departments[dept] = departments.get(dept, 0) + carbon
return {k: round(v, 3) for k, v in departments.items()}

```

API Endpoint:

```

# ml_services/ocr/api.py

@app.post("/ocr/receipts/bulk")
async def process_bulk_receipts(
    zip_file: UploadFile = File(...),
    organization_id: str = Form(...),
    uploaded_by: str = Form(...)
):
    """
    Upload zip of receipts for batch processing.
    Returns organization-level summary + individual results.
    """
    with tempfile.TemporaryDirectory() as tmpdir:
        # Extract zip
        zip_path = Path(tmpdir) / "receipts.zip"
        with open(zip_path, 'wb') as f:
            shutil.copyfileobj(zip_file.file, f)

        with zipfile.ZipFile(zip_path) as z:
            z.extractall(tmpdir)

        # Get all image files
        receipt_paths = [
            str(p) for p in Path(tmpdir).rglob('*')
            if p.suffix.lower() in ['.jpg', '.jpeg', '.png', '.pdf']
        ]

        # Process in batch
        result = process_receipt_batch(receipt_paths, organization_id, uploaded_by)

    return JSONResponse(result)

```

Target Metrics: - Process 500 receipts in <5 minutes (parallel processing)
 - Same field extraction accuracy as single-receipt (>88% F1) - Organization summary accuracy >95%

1.2 Organization-Level Receipt Analytics Use Case: Admin sees “Engineering department spent 50K on high-carbon items this month”

Technical Implementation:

```
# ml_services/ocr/org_analytics.py
"""
Organization-level analytics from aggregated receipt data.
No new ML models - just smart aggregation of individual outputs.
"""

def generate_org_receipt_report(
    organization_id: str,
    start_date: str,
    end_date: str
) -> dict:
    """
    Generate organization-wide receipt analytics.
    Called by backend for admin dashboard.
    """

    # In production: Query database for all receipts in date range
    # Here: Simplified example

    receipts = query_receipts_by_org(organization_id, start_date, end_date)

    # Aggregate metrics
    total_carbon = sum(r['carbon_kg'] for r in receipts)
    total_receipts = len(receipts)

    # Category breakdown
    categories = {}
    for r in receipts:
        cat = r.get('category', 'unknown')
        categories[cat] = categories.get(cat, 0) + r['carbon_kg']

    # Department leaderboard (for gamification)
    dept_emissions = aggregate_by_department(receipts)
    dept_leaderboard = sorted(
        [{"department": k, "carbon_kg": v} for k, v in dept_emissions.items()],
        key=lambda x: x['carbon_kg']
    )

    # Top carbon items across organization
    all_items = []
    for r in receipts:
        all_items.extend(r.get('items', []))
```

```

top_items = sorted(
    all_items,
    key=lambda x: x.get('carbon_kg', 0),
    reverse=True
)[:20]

# Trends (compare with previous period)
prev_period_carbon = query_previous_period_carbon(
    organization_id, start_date, end_date
)
trend_pct = ((total_carbon - prev_period_carbon) / max(prev_period_carbon, 1)) * 100

return {
    "organization_id": organization_id,
    "period": {"start": start_date, "end": end_date},
    "summary": {
        "total_carbon_kg": round(total_carbon, 2),
        "total_receipts": total_receipts,
        "avg_per_receipt_kg": round(total_carbon / max(total_receipts, 1), 3),
        "trend_vs_previous_pct": round(trend_pct, 1)
    },
    "category_breakdown": categories,
    "department_leaderboard": dept_leaderboard, # For gamification
    "top_carbon_items": top_items,
    "recommendations": generate_org_recommendations(categories, top_items)
}

def generate_org_recommendations(categories: dict, top_items: list) -> list:
    """Generate organization-level reduction recommendations."""
    recs = []

    # If food category is high
    if categories.get('food', 0) > 500: # >500kg/month
        recs.append({
            "type": "policy",
            "title": "High food emissions detected",
            "suggestion": "Consider implementing Meat-Free Mondays in office cafeteria",
            "potential_reduction_kg": categories['food'] * 0.2, # 20% reduction
            "effort": "medium"
        })

    # If transport is high
    if categories.get('transport', 0) > 1000:
        recs.append({
            "type": "policy",

```

```

        "title": "High transport reimbursements",
        "suggestion": "Incentivize public transport with 1000/month allowance",
        "potential_reduction_kg": categories['transport'] * 0.15,
        "cost_savings_annual": 50000
    })

    return recs

```

TRACK 2: Food Recognition (Enhanced for Teams)

Original Features (Kept)

- EfficientNet-B4 on Food-101
- Indian food fine-tuning
- Individual meal logging

New B2B2C Features (Added)

2.1 Cafeteria Mode (Batch Food Recognition) Use Case: Office cafeteria has 5 dishes today. HR uploads photos once. All employees select from those 5 when logging lunch.

Technical Implementation:

```

# ml_services/food/cafeteria_mode.py
"""
Pre-process cafeteria menu for the day.
Employees then select from pre-recognized dishes (faster, no redundant inference).
"""

def create_cafeteria_menu(
    organization_id: str,
    date: str,
    dish_images: List[UploadFile],
    location: str = "main_cafeteria"
) -> dict:
    """
    Admin uploads photos of today's cafeteria dishes.
    System recognizes them once, stores as menu.
    Employees select from menu (no need to re-recognize).
    """

    from .inference import predict_food

    menu_items = []

    for img_file in dish_images:

```

```

        # Save temporarily
        with tempfile.NamedTemporaryFile(delete=False, suffix='.jpg') as tmp:
            tmp.write(img_file.file.read())
            tmp_path = tmp.name

        # Recognize once
        prediction = predict_food(tmp_path, top_k=1)
        top_pred = prediction['predictions'][0]

        menu_items.append({
            "dish_name": top_pred['food'],
            "carbon_per_100g_kg": top_pred['carbon_per_100g_kg'],
            "confidence": top_pred['confidence'],
            "image_url": upload_to_s3(tmp_path) # Store image
        })

        os.unlink(tmp_path)

        # Store in database (cafeteria_menus table)
        menu_id = store_cafeteria_menu(
            organization_id=organization_id,
            date=date,
            location=location,
            items=menu_items
        )

        return {
            "menu_id": menu_id,
            "organization_id": organization_id,
            "date": date,
            "location": location,
            "items": menu_items,
            "usage_instructions": "Employees select from this menu when logging meals"
        }

def log_employee_cafeteria_meal(
    user_id: str,
    menu_id: str,
    selected_dishes: List[dict] # [{"dish_name": "Biryani", "serving_g": 250}, ...]
) -> dict:
    """
    Employee logs meal by selecting from pre-recognized cafeteria menu.
    No ML inference needed - just lookup carbon values.
    """
    menu = get_cafeteria_menu(menu_id)

```

```

total_carbon = 0
meal_items = []

for selection in selected_dishes:
    dish_name = selection['dish_name']
    serving_g = selection.get('serving_g', 100)

    # Find in menu
    dish = next((d for d in menu['items'] if d['dish_name'] == dish_name), None)
    if not dish:
        continue

    # Calculate carbon
    carbon_kg = dish['carbon_per_100g_kg'] * (serving_g / 100)
    total_carbon += carbon_kg

    meal_items.append({
        "dish": dish_name,
        "serving_g": serving_g,
        "carbon_kg": round(carbon_kg, 4)
    })

# Store in user's carbon transactions
transaction_id = store_food_transaction(
    user_id=user_id,
    items=meal_items,
    total_carbon_kg=total_carbon,
    source="cafeteria_menu"
)

return {
    "transaction_id": transaction_id,
    "total_carbon_kg": round(total_carbon, 4),
    "items": meal_items,
    "comparison": {
        "vs_non_veg_meal": round((1.5 - total_carbon), 3), # Typical non-veg = 1.5kg
        "trees_to_offset": round(total_carbon / 0.025, 1) # Neem absorbs ~25kg/year
    }
}

```

Benefits: - 50× faster for employees (no photo upload, no ML inference) - Consistent carbon values across organization (same dish = same carbon) - Enables cafeteria-level optimization (what dishes to serve)

2.2 Team Food Challenge System Use Case: “Engineering vs Sales: Who can eat lower-carbon meals for 1 month?”

Technical Implementation:

```
# ml_services/food/team_challenges.py
"""
Team-based food carbon challenges.
Aggregates individual food logs → team totals → leaderboard.
"""

def create_team_challenge(
    organization_id: str,
    challenge_name: str,
    teams: List[str], # ["engineering", "sales", "hr"]
    start_date: str,
    end_date: str,
    goal_type: str = "lowest_total" # or "highest_reduction_pct"
) -> dict:
    """
    Admin creates a team challenge.
    System automatically tracks food logs for team members.
    """

    challenge_id = generate_uuid()

    # Store challenge config in database
    store_challenge({
        "challenge_id": challenge_id,
        "organization_id": organization_id,
        "name": challenge_name,
        "teams": teams,
        "start_date": start_date,
        "end_date": end_date,
        "goal_type": goal_type,
        "status": "active"
    })

    return {
        "challenge_id": challenge_id,
        "message": f"Challenge '{challenge_name}' created. Employees' food logs will auto-co
        "leaderboard_url": f"/api/v1/challenges/{challenge_id}/leaderboard"
    }

def get_challenge_leaderboard(challenge_id: str) -> dict:
    """
    Real-time leaderboard for team challenge.
    """
```

```

Called by frontend to display standings.
"""
challenge = get_challenge(challenge_id)

# Query all food transactions in date range for this org
transactions = query_food_transactions(
    organization_id=challenge['organization_id'],
    start_date=challenge['start_date'],
    end_date=challenge['end_date']
)

# Aggregate by team
team_totals = {}
team_member_counts = {}

for txn in transactions:
    user = get_user(txn['user_id'])
    team = user.get('department', 'unassigned')

    if team not in challenge['teams']:
        continue

    team_totals[team] = team_totals.get(team, 0) + txn['carbon_kg']
    team_member_counts[team] = team_member_counts.get(team, set())
    team_member_counts[team].add(txn['user_id'])

# Calculate per-capita (fairer for different team sizes)
leaderboard = []
for team in challenge['teams']:
    total = team_totals.get(team, 0)
    members = len(team_member_counts.get(team, []))
    per_capita = total / max(members, 1)

    leaderboard.append({
        "team": team,
        "total_carbon_kg": round(total, 2),
        "members_participating": members,
        "per_capita_kg": round(per_capita, 3),
        "rank": 0 # Will be assigned after sorting
    })

# Sort by goal type
if challenge['goal_type'] == 'lowest_total':
    leaderboard.sort(key=lambda x: x['per_capita_kg'])

for rank, team in enumerate(leaderboard, 1):

```

```

team['rank'] = rank

return {
    "challenge_id": challenge_id,
    "challenge_name": challenge['name'],
    "period": {
        "start": challenge['start_date'],
        "end": challenge['end_date']
    },
    "leaderboard": leaderboard,
    "last_updated": datetime.now().isoformat()
}

```

Gamification Features: - Real-time leaderboards (updated every food log) - Team badges (“Veggie Champions”, “Low-Carbon Heroes”) - Department challenges (automatic enrollment) - Monthly winner announcements

Engineering Scope: - No new ML models (uses existing food recognition) - Pure aggregation + leaderboard logic - Fits within project scope (2-3 days implementation)

TRACK 3: Policy NLP (Already B2B-Focused - Minor Tweaks)

Original Features (Kept)

- Budget 2026 extraction
- Clause classification
- Sector tagging
- CCUS flagging

Minor Enhancement: Organization Policy Mapping

```

# ml_services/policy/org_mapper.py
"""
Map extracted policy clauses to specific organizations based on:
- Organization sector (cement, steel, etc.)
- Organization size (annual emissions threshold)
- Organization location (state-specific regulations)
"""

def get_applicable_policies(
    organization_id: str
) -> dict:
    """
    For a given organization, return all applicable policy clauses.
    """

```

```

Backend calls this to populate compliance dashboard.
"""
org = get_organization(organization_id)

# Query policy_clauses table with filters
applicable = query_clauses(
    sectors=org['sectors'],
    organization_sizes=['sme', 'large'] if org['annual_emissions_tonnes'] > 10000 else
    geographic_scope=org['location_state']
)

# Priority scoring
for clause in applicable:
    clause['priority_score'] = calculate_priority(clause, org)

applicable.sort(key=lambda c: c['priority_score'], reverse=True)

return {
    "organization_id": organization_id,
    "total_applicable_clauses": len(applicable),
    "high_priority": [c for c in applicable if c['priority_score'] >= 8],
    "medium_priority": [c for c in applicable if 5 <= c['priority_score'] < 8],
    "low_priority": [c for c in applicable if c['priority_score'] < 5],
    "all_clauses": applicable
}

```

No new ML work - just filtering logic on existing policy extraction.

TRACK 4: Forecasting + Analytics (Org-Level Outputs Added)

Original Features (Kept)

- Prophet forecasting (individual)
- Anomaly detection
- SHAP explainability

New B2B2C Features (Added)

4.1 Organization-Level Forecasting Use Case: “HR dashboard shows: ‘Organization projected to emit 50 tonnes this quarter’ ”

Technical Implementation:

```

# ml_services/forecasting/org_forecaster.py
"""

```

```

Organization-level carbon forecasting.
Same Prophet model, just trained on aggregated org data.
"""

def forecast_organization_emissions(
    organization_id: str,
    horizon_days: int = 90
) -> dict:
    """
    Forecast organization-wide emissions.
    Aggregates all employees' historical data + train Prophet + forecast.
    """

    # Query all carbon transactions for this organization
    org_transactions = query_org_transactions(organization_id)

    # Aggregate by day
    daily_totals = aggregate_by_day(org_transactions)

    # Prepare Prophet format
    df = pd.DataFrame(daily_totals)
    df.columns = ['ds', 'y'] # Prophet requires these names

    # Train Prophet model (same as individual, just on org data)
    from ml_services.forecasting.forecaster import CarbonForecaster
    forecaster = CarbonForecaster()
    forecaster.fit(df)

    # Forecast
    forecast = forecaster.forecast(horizon_days=horizon_days)

    # Add organization context
    forecast['organization_id'] = organization_id
    forecast['employees_contributing'] = get_active_employees_count(organization_id)
    forecast['per_employee_forecast_kg'] = (
        forecast['total_predicted_kg'] / forecast['employees_contributing']
    )

    # Department breakdown forecast (if enough data)
    dept_forecasts = forecast_by_department(organization_id, horizon_days)
    forecast['department_forecasts'] = dept_forecasts

    return forecast

def forecast_by_department(organization_id: str, horizon_days: int) -> dict:
    """

```

```

Forecast for each department separately.
Useful for: "Engineering projected to increase 15%, Sales to decrease 10%"
"""
departments = get_org_departments(organization_id)
dept_forecasts = {}

for dept in departments:
    dept_data = query_dept_transactions(organization_id, dept)

    if len(dept_data) < 30: # Need at least 30 days data
        continue

    daily = aggregate_by_day(dept_data)
    df = pd.DataFrame(daily)
    df.columns = ['ds', 'y']

    forecaster = CarbonForecaster()
    forecaster.fit(df)

    forecast = forecaster.forecast(horizon_days=14) # 2-week lookahead

    dept_forecasts[dept] = {
        "total_predicted_kg": forecast['total_predicted_kg'],
        "trend": forecast['trend'],
        "daily_average_kg": forecast['daily_average_kg']
    }

return dept_forecasts

```

4.2 Department Comparison Analytics Use Case: “Which department has highest per-capita emissions?”

```

# ml_services/analytics/org_analytics.py
"""
Organization-level analytics aggregations.
NOT new ML models - just smart SQL + pandas aggregations.
"""

def generate_department_comparison(
    organization_id: str,
    start_date: str,
    end_date: str
) -> dict:
    """

```

```

Compare departments on multiple carbon metrics.
For admin dashboard: "See which teams are doing well"
"""
departments = get_org_departments(organization_id)

comparisons = []

for dept in departments:
    dept_data = query_dept_transactions(
        organization_id, dept, start_date, end_date
    )

    total_carbon = sum(t['carbon_kg'] for t in dept_data)
    member_count = get_dept_member_count(organization_id, dept)
    per_capita = total_carbon / max(member_count, 1)

    # Category breakdown
    categories = aggregate_by_category(dept_data)
    top_category = max(categories, key=categories.get) if categories else "unknown"

    # Trend
    prev_period = query_previous_period(organization_id, dept, start_date, end_date)
    trend_pct = ((total_carbon - prev_period) / max(prev_period, 1)) * 100

    comparisons.append({
        "department": dept,
        "total_carbon_kg": round(total_carbon, 2),
        "members": member_count,
        "per_capita_kg": round(per_capita, 3),
        "top_category": top_category,
        "trend_vs_previous_pct": round(trend_pct, 1),
        "trend_direction": "increasing" if trend_pct > 5 else "decreasing" if trend_pct
    })

# Sort by per-capita (fairest comparison)
comparisons.sort(key=lambda d: d['per_capita_kg'])

# Assign rankings
for rank, dept in enumerate(comparisons, 1):
    dept['rank'] = rank
    dept['performance'] = "excellent" if rank <= len(comparisons) * 0.2 else \
        "good" if rank <= len(comparisons) * 0.5 else \
        "needs_improvement"

return {
    "organization_id": organization_id,

```

```

    "period": {"start": start_date, "end": end_date},
    "departments": comparisons,
    "organization_average_per_capita_kg": round(
        sum(d['total_carbon_kg'] for d in comparisons) /
        sum(d['members'] for d in comparisons), 3
    ),
    "best_performing_department": comparisons[0]['department'],
    "most_improvement_needed": comparisons[-1]['department']
}

```

Output Used By: - Admin dashboard (backend calls this API) - Monthly email reports to HR - Gamification (department leaderboards)

TRACK 5: Multi-Tenant ML Infrastructure (NEW)

Purpose: Ensure data isolation between organizations in all ML pipelines

Scope: Engineering infrastructure, NOT new ML models

5.1 Multi-Tenant Data Isolation Layer

```

# ml_services/common/multi_tenant.py
"""
Multi-tenant wrapper for all ML services.
Ensures organization A can't access organization B's data or models.
"""

from functools import wraps
from typing import Callable
import jwt

def require_organization_context(f: Callable):
    """
    Decorator that enforces organization_id is present and valid.
    All multi-tenant ML endpoints must use this.
    """
    @wraps(f)
    async def wrapper(*args, **kwargs):
        # Extract organization_id from request
        organization_id = kwargs.get('organization_id')
        user_id = kwargs.get('user_id')

        if not organization_id:
            raise ValueError("organization_id required for multi-tenant operation")

        # Verify user has access to this organization

```

```

        if not verify_user_org_access(user_id, organization_id):
            raise PermissionError(f"User {user_id} not authorized for org {organization_id}")

        # Proceed with validated context
        return await f(*args, **kwargs)

    return wrapper

def filter_by_organization(query_results: list, organization_id: str) -> list:
    """
    Filter ML outputs to only include data from specified organization.
    Defense-in-depth: even if query leaks, this catches it.
    """
    return [
        r for r in query_results
        if r.get('organization_id') == organization_id or
        r.get('user', {}).get('organization_id') == organization_id
    ]

# Apply to all org-level ML endpoints
@app.post("/ml/forecasting/organization")
@require_organization_context
async def forecast_organization_wrapper(
    organization_id: str,
    user_id: str,
    horizon_days: int
):
    """
    Organization forecast with multi-tenant enforcement.
    """
    result = forecast_organization_emissions(organization_id, horizon_days)

    # Double-check result only contains this org's data
    assert result['organization_id'] == organization_id

    return result

```

5.2 Role-Based Feature Access

```

# ml_services/common/rbac.py
"""
Role-based access control for ML features.

```

Different features available based on user role.
"""

```
ROLE_PERMISSIONS = {
    "employee": [
        "log_own_emissions",
        "view_own_analytics",
        "view_team_leaderboard",
        "join_challenges"
    ],
    "manager": [
        "log_own_emissions",
        "view_own_analytics",
        "view_team_leaderboard",
        "view_department_analytics",
        "create_department_challenge"
    ],
    "admin": [
        "log_own_emissions",
        "view_own_analytics",
        "view_all_org_analytics",
        "create_org_challenge",
        "bulk_upload_receipts",
        "create_cafeteria_menu",
        "view_compliance_dashboard",
        "generate_reports"
    ],
    "free_tier": [
        "log_own_emissions",
        "view_own_analytics",
        "view_public_leaderboard"
    ]
}

def check_permission(user_role: str, required_permission: str) -> bool:
    """Check if user role has required permission."""
    return required_permission in ROLE_PERMISSIONS.get(user_role, [])

def require_permission(permission: str):
    """Decorator for endpoints requiring specific permission."""
    def decorator(f: Callable):
        @wraps(f)
        async def wrapper(*args, **kwargs):
            user = kwargs.get('user')

```

```

        if not user:
            raise PermissionError("User context required")

        if not check_permission(user['role'], permission):
            raise PermissionError(
                f"Permission '{permission}' required. User has role '{user['role']}'"
            )

        return await f(*args, **kwargs)
    return wrapper
return decorator

# Example usage
@app.post("/ml/ocr/receipts/bulk")
@require_permission("bulk_upload_receipts")
async def bulk_upload_endpoint(...):
    """Only admins can bulk upload."""
    pass

@app.get("/ml/analytics/organization")
@require_permission("view_all_org_analytics")
async def org_analytics_endpoint(...):
    """Only admins see full org analytics."""
    pass

```

TRACK 6: RL Behavioral Nudging (Team-Aware)

Original Features (Kept)

- DQN agent
- 8-dimensional state space
- 5 nudge types

Enhancement: Add Team Context to State

```

# ml_services/rl/dqn_agent.py (modified)

# OLD STATE (8 features - individual only):
# [current_carbon, trend, last_nudge_type, days_since_nudge,
#  engagement_score, eco_awareness, time_of_day, day_of_week]

# NEW STATE (11 features - individual + team context):
STATE_DIM = 11

```

```

def extract_state_features(user_data: dict, team_data: dict) -> np.ndarray:
    """
    Extract RL state features with team context.

    Args:
        user_data: Individual user stats
        team_data: User's team/department stats

    Returns:
        11-dimensional state vector
    """
    return np.array([
        # Individual features (8)
        normalize_carbon(user_data['current_carbon_kg']),
        user_data['trend'], # -1/0/1
        user_data['last_nudge_type'] / 4.0, # Normalize 0-4 → 0-1
        min(user_data['days_since_last_nudge'] / 7.0, 1.0),
        user_data['engagement_score'],
        user_data['eco_awareness'],
        user_data['time_of_day'] / 24.0,
        user_data['day_of_week'] / 7.0,

        # NEW: Team context (3)
        normalize_carbon(team_data.get('team_avg_carbon', 0)),
        team_data.get('team_rank', 0.5), # 0-1, where 0=best team, 1=worst
        team_data.get('team_challenge_active', 0) # 0/1 binary
    ], dtype=np.float32)

# NEW ACTION: Team-aware nudges
ACTION_DIM = 6 # Added one more action

ACTIONS = {
    0: "no_nudge",
    1: "gentle_reminder",
    2: "comparison_vs_peers",
    3: "goal_progress",
    4: "challenge_invite",
    5: "team_comparison" # NEW: "Your team is 2nd place! Help close the gap"
}

def generate_nudge_text(action: int, user_data: dict, team_data: dict) -> str:
    """Generate personalized nudge based on action and context."""
    if action == 0:

```

```

        return None

    elif action == 5: # Team comparison (new)
        team_rank = team_data.get('team_rank_position', 0)
        team_name = team_data.get('team_name', 'Your team')

        if team_rank == 1:
            return f" {team_name} is #1 in low-carbon! Keep it up!"
        elif team_rank <= 3:
            return f" {team_name} is #{team_rank}. So close to #1!"
        else:
            return f"{team_name} is #{team_rank}. Small actions = big climb!"

    # ... other actions remain same

```

Why This Works: - Team context creates social pressure (proven effective in research) - RL agent learns when team-based nudges work better than individual
 - Engineering scope: Just 3 extra state features + 1 extra action

TRACK 7: Admin Analytics APIs (Semester 2)

Purpose: Provide aggregated analytics endpoints for admin dashboards

Scope: Pure aggregation code, NOT new ML models

7.1 Organization KPI Dashboard API

```

# ml_services/analytics/admin_api.py
"""
Admin analytics endpoints.
Backend calls these to populate HR/admin dashboards.
"""

@app.get("/ml/analytics/organization/kpis")
@require_permission("view_all_org_analytics")
async def get_organization_kpis(
    organization_id: str,
    period: str = "month" # week | month | quarter | year
):
    """
    Key performance indicators for organization dashboard.

    Returns:
    {
        "total_carbon_kg": 15000,
        "active_users": 250,
    }
    """

```

```

        "per_capita_kg": 60,
        "trend_vs_previous": -12.5, # Negative = improvement
        "category_breakdown": {...},
        "department_summary": {...},
        "top_performers": [...],
        "compliance_status": {...}
    }
    """
    start_date, end_date = parse_period(period)

    # Query all transactions
    transactions = query_org_transactions(organization_id, start_date, end_date)

    # Calculate KPIs (pure aggregation)
    total_carbon = sum(t['carbon_kg'] for t in transactions)
    active_users = len(set(t['user_id'] for t in transactions))
    per_capita = total_carbon / max(active_users, 1)

    # Previous period for trend
    prev_transactions = query_previous_period_transactions(
        organization_id, start_date, end_date
    )
    prev_total = sum(t['carbon_kg'] for t in prev_transactions)
    trend_pct = ((total_carbon - prev_total) / max(prev_total, 1)) * 100

    # Category breakdown
    categories = aggregate_by_category(transactions)

    # Department summary
    departments = generate_department_comparison(
        organization_id, start_date, end_date
    )

    # Top performers (lowest per-capita in org)
    user_stats = calculate_user_stats(transactions)
    top_performers = sorted(user_stats, key=lambda u: u['per_capita_kg'])[:10]

    # Compliance status (from Policy NLP)
    compliance = get_compliance_status(organization_id)

    return {
        "organization_id": organization_id,
        "period": {"start": start_date, "end": end_date},
        "kpis": {
            "total_carbon_kg": round(total_carbon, 2),
            "active_users": active_users,

```

```

        "per_capita_kg": round(per_capita, 3),
        "trend_vs_previous_pct": round(trend_pct, 1),
        "trend_direction": "improving" if trend_pct < 0 else "worsening"
    },
    "category_breakdown": categories,
    "department_summary": departments,
    "top_performers": top_performers,
    "compliance_status": compliance,
    "last_updated": datetime.now().isoformat()
}

```

7.2 Export & Reporting API

```

@app.post("/ml/analytics/organization/export")
@require_permission("generate_reports")
async def export_organization_report(
    organization_id: str,
    start_date: str,
    end_date: str,
    format: str = "pdf" # pdf / excel / csv
):
    """
    Generate exportable report for HR/management.
    Includes: executive summary, trends, department breakdown, recommendations.
    """
    # Gather all analytics
    kpis = await get_organization_kpis(organization_id, f"{start_date}_{end_date}")
    forecast = forecast_organization_emissions(organization_id, horizon_days=30)

    # Generate report document
    if format == "pdf":
        pdf_path = generate_pdf_report(kpis, forecast)
        return FileResponse(pdf_path, media_type="application/pdf")

    elif format == "excel":
        excel_path = generate_excel_report(kpis, forecast)
        return FileResponse(excel_path, media_type="application/vnd.ms-excel")

    else: # csv
        csv_data = generate_csv_export(kpis)
        return Response(csv_data, media_type="text/csv")

```

UPDATED TIMELINE (48 Weeks with Hybrid Features)

Semester 1 (Weeks 1-24)

Weeks 7-8: Receipt OCR with Bulk Processing

- **Original:** Individual receipt OCR + NER
- **Added:** Bulk upload API (2 days), org analytics (1 day)
- **Total:** 5 days (was 3)

Weeks 9-10: Food Recognition with Cafeteria Mode

- **Original:** Food-101 + Indian food training
- **Added:** Cafeteria menu API (1 day), team challenges (1 day)
- **Total:** 6 days (was 4)

Weeks 11-12: Policy NLP (Minimal Changes)

- **Original:** Clause extraction, classification, sector tagging
- **Added:** Organization policy mapping (4 hours)
- **Total:** 4 days (same)

Weeks 13-16: Forecasting + Organization Analytics

- **Original:** Prophet forecasting, anomaly detection
- **Added:** Org forecasting (1 day), department comparison (1 day)
- **Total:** 7 days (was 5)

Weeks 17-18: Multi-Tenant Infrastructure (NEW)

- Data isolation layer (2 days)
- Role-based access control (2 days)
- Testing multi-tenancy (1 day)
- **Total:** 5 days

Weeks 19-20: XAI + RL with Team Context

- **Original:** SHAP, GradCAM, DQN agent
- **Added:** Team-aware state features (1 day)
- **Total:** 5 days (was 3)

Weeks 21-24: Testing & Documentation

- Integration testing (all services)
- Multi-tenant security testing
- Documentation
- **Total:** 4 weeks (same)

Semester 2 (Weeks 25-48)

Weeks 25-30: Advanced Features

- FinBERT transaction categorization (3 days)
- Advanced RL training on real data (5 days)
- Mobile app with team features (8 days)

Weeks 31-36: Enterprise & Admin Features

- Admin analytics APIs (5 days)
- Export & reporting (3 days)
- Supply chain mapping (already B2B)
- Blockchain integration (individual + org)

Weeks 37-42: Scaling & Optimization

- Performance optimization
- K8s auto-scaling
- Multi-region deployment

Weeks 43-48: Pilots, Testing, Launch

- Pilot with 2-3 SMEs (20-50 employees each)
- Real user testing (100+ individual users)
- Final documentation
- Presentation preparation

SCOPE CONTROL: WHAT WE'RE NOT ADDING

To keep this an engineering project (not business project), we're NOT building:

Billing/payment systems (assume backend handles this) **Complex admin UIs** (ML provides APIs, frontend uses them) **HR management features** (we aggregate data, not manage employees) **Calendar integrations** (out of scope) **Email marketing** (out of scope) **Customer support** (out of scope) **Advanced supply chain tracing** (keep it simple - just Scope 1, 2, 3)

What we ARE building: Multi-tenant ML infrastructure Organization-level aggregation APIs Team/department analytics endpoints Bulk processing capabilities Role-based feature access Same ML models with dual outputs (individual + org)

EVALUATION METRICS (Updated)

Individual Features (B2C)

- Receipt OCR field F1 > 88%
- Food recognition Top-1 > 85%
- Forecasting MAPE < 10%
- RL engagement lift > 30%

Organization Features (B2B2C)

- Bulk processing: 500 receipts in <5 minutes
- Org forecast accuracy: MAPE < 15% (harder than individual)
- Department analytics latency: <2 seconds
- Multi-tenant isolation: 100% (zero data leaks)
- Team challenge participation: >60% of employees in pilot orgs

System Performance

- API response time: <200ms (95th percentile)
 - Concurrent users: 10,000+
 - Multi-tenant queries: No cross-org data leakage (security audit)
-

TEAM RESPONSIBILITIES (Updated)

Member 1 (Frontend - You collaborate with them)

New Additions: - Admin dashboard consuming your analytics APIs - Team leaderboard UI - Cafeteria menu selector - Bulk upload interface - Department comparison charts

Member 2 (Backend - You collaborate with them)

New Additions: - Multi-tenant database RLS (Row Level Security) - Organization/department tables - User role management - Calling your ML APIs with org context

Member 3 (ML/AI - YOU)

Your Responsibilities:

Week 7-8 (Receipt OCR): - Individual OCR pipeline (same as before) - **NEW:** Bulk processing API - **NEW:** Organization-level aggregation

Week 9-10 (Food Recognition): - Food-101 training (same as before) - **NEW:** Cafeteria mode API - **NEW:** Team challenge aggregation

Week 11-12 (Policy NLP): - Clause extraction (same as before) - **NEW:** Organization policy filtering (4 hours)

Week 13-16 (Forecasting): - Prophet model (same as before) - **NEW:** Organization forecasting - **NEW:** Department analytics

Week 17-18 (Multi-Tenant): - **NEW:** Data isolation decorators - **NEW:** RBAC implementation - **NEW:** Security testing

Week 19-20 (RL + XAI): - RL agent (mostly same) - **NEW:** Team-aware state features - XAI (same as before)

Semester 2: - Admin analytics APIs - Report generation - Pilot support - Performance optimization

FINAL RECOMMENDATIONS

DO THIS

1. **Follow this restructured plan** - adds B2B2C features without bloating scope
2. **Reuse ML models** - same models, multiple output formats
3. **Focus on APIs** - let backend/frontend handle UIs
4. **Test multi-tenancy rigorously** - data leaks = project failure
5. **Document organization features** - show you thought about real-world deployment

DON'T DO THIS

1. **Don't build separate B2C and B2B ML systems** - massive scope explosion
2. **Don't add business logic** - stick to technical ML engineering
3. **Don't build admin dashboards yourself** - provide APIs only
4. **Don't add billing/payments** - out of scope for engineering project
5. **Don't gold-plate** - if a feature takes >3 days, it's too much

CONCLUSION: PROJECT REMAINS STRONG

You're adding ~15% more work for 300% more value.

Original plan: Pure individual ML features (good engineering project) **Updated plan:** Individual ML + organization aggregations (excellent engineering project with real-world applicability)

Time added: ~2 weeks across entire 48-week timeline **Value added:** Makes project commercially viable, demonstrates system design thinking, shows you understand deployment realities

For your panel: “We built sophisticated multi-modal AI (OCR, Food Recognition, NLP, Forecasting, RL, XAI) with multi-tenant architecture supporting both individual users and organizations. The ML models are the same - we architected the system to serve multiple use cases from one codebase.”

This is what senior engineers do - build systems that scale beyond initial requirements. Your panel will recognize this as mature thinking.

IMMEDIATE NEXT STEPS (This Week)

1. **Start Receipt OCR (individual version)** - follow original guide from previous plan
2. **While OCR trains:** Design org analytics API contracts (write the function signatures)
3. **By end of week:** Have individual receipt OCR working + bulk processing API spec written
4. **Next week:** Implement bulk processing while food model trains on Colab

You're ready to build. This plan is solid, scoped, and impressive.